Cambio en la estructura de módulos para cumplir con las restricciones del proyecto

SolicitudAmistad

SolicitudAmistad.h <Select Symbol>

```h
#ifndef SOLICITUDAMISTAD_H
#define SOLICITUDAMISTAD_H

#include "cadena.h"

class Usuario; // Declaracion adelantada para evitar dependencias circulares

class SolicitudAmistad
{
private:
    Cadena fechaSolicitud;
    int estado; // 0: Pendiente, 1: Aceptada, 2: Rechazada
    Usuario* emisor;
    Usuario* receptor;

public:
    SolicitudAmistad();
    SolicitudAmistad(const Cadena& fecha, Usuario* em, Usuario* rec);
    SolicitudAmistad(const SolicitudAmistad& otra);
    ~SolicitudAmistad();

    SolicitudAmistad& operator=(const SolicitudAmistad& otra);

    void aceptar();
    void rechazar();

    // Getters necesarios para que RedLeon/RedSocial pueda filtrar y consultar
    int getEstado() const;
    const Cadena& getFechaSolicitud() const;
    Usuario* getEmisor() const;
    Usuario* getReceptor() const;
};

#endif // SOLICITUDAMISTAD_H
```

Nota: Se realizan cambios al diseño original por el **Uso de #include <string> y string:** Prohibido por la restricción de cero STL. Debe sustituirse por Cadena.

Falta de actores (emisor y receptor): La solicitud está "flotando en el aire"; no sabe quién la envió ni a quién va dirigida. Para que el usuario pueda ver las solicitudes que ha recibido o saber a quién le mandó una, necesita punteros a Usuario (emisor y receptor).

Tipo del atributo estado: Manejarlo como string ("Pendiente", "Aceptada") consume memoria innecesaria y hace comparaciones lentas. Se optimiza cambiándolo a un entero int (0: Pendiente, 1: Aceptada, 2: Rechazada).

Nombres alineados al diagrama oficial: Los métodos en el diagrama aprobado son aceptar() y rechazar().

MENSAJE .H

mensaje.h <Select Symbol>

```cpp
#ifndef MENSAJE_H
#define MENSAJE_H

#include "cadena.h"

class Usuario; // Declaracion adelantada

class Mensaje
{
private:
    Cadena contenido;
    Cadena fechaEnvio;
    bool leido;
    Usuario* emisor;
    Usuario* receptor;

public:
    Mensaje();
    Mensaje(const Cadena& cont, const Cadena& fecha, Usuario* em, Usuario* rec);
    Mensaje(const Mensaje& otro);
    ~Mensaje();

    Mensaje& operator=(const Mensaje& otro);

    void marcarLeido();
    void mostrarMensaje() const;

    bool esLeido() const;
    const Cadena& getFechaEnvio() const;
    const Cadena& getContenido() const;
    Usuario* getEmisor() const;
    Usuario* getReceptor() const;
};

#endif // MENSAJE_H
```

NOTA: Se realizan cambios al diseño original por el **Uso de #include <string> y string:** El enunciado prohíbe el uso de contenedores de la STL. Debe migrarse a la clase propia Cadena.

Ausencia de emisor y receptor: El mensaje solo almacena texto y fecha, por lo que el sistema no sabe quién lo envió ni quién debe recibirlo, impidiendo la lectura de nuevos mensajes para un usuario

específico y la búsqueda por fecha de mensajes recibidos. Requiere los punteros Usuario* emisor y Usuario* receptor.

Responsabilidad del método enviarMensaje(): En tu clase preliminar, enviarMensaje() solo imprimía un texto en consola (cout << "Mensaje enviado" << endl;). El envío real consiste en validar longitud (hasta 512 caracteres), instanciar el mensaje y agregarlo dinámicamente a la colección del sistema, responsabilidad que corresponde al controlador (RedLeon).

Falta de constructor de copia y sobrecarga de operador: Necesarios para la gestión segura de memoria dinámica en arreglos.

Conclusión de cambios

Se realizan cambios en los módulos por las restricciones del trabajo **Motivo:** En el diseño preliminar, la relación de amistad se representaba con una clase Amistad independiente. Sin embargo, el enunciado del proyecto requiere explícitamente optimizar la búsqueda y cálculo de amigos en común para más de 1000 usuarios mediante una **matriz de adyacencia** (bool**).

Acción tomada: Se retira la clase Amistad (y la entidad pasiva sugerencia). Las relaciones de amistad ahora se modelan de forma bidireccional y directa en la matriz de adyacencia de la clase controladora (RedLeon). Esto reduce la sobrecarga de memoria dinámica por objeto y agiliza las consultas en tiempo constante $O(1)$